In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plot
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# Path to the downloaded CSV file
file_path = r"C:\Users\Difo\Documents\data science\The-Applied-Artificial-Intellige

# Load the dataset
data = pd.read_csv(file_path)

# Display the first five rows
print(data.head())

# Display dataset dimensions
print("Dataset shape:", data.shape)

# Display column names
print("Columns:", data.columns.tolist())
```

```
  buying maintenance doors persons luggage_boot safety  class
0  vhigh       vhigh     2       2        small    low  unacc
1  vhigh       vhigh     2       2        small    med  unacc
2  vhigh       vhigh     2       2        small   high  unacc
3  vhigh       vhigh     2       2          med    low  unacc
4  vhigh       vhigh     2       2          med    med  unacc
Dataset shape: (1728, 7)
Columns: ['buying', 'maintenance', 'doors', 'persons', 'luggage_boot', 'safety', 'cl
ass']
```

In [2]:
```
# Create a copy of the original dataset
data_encoded = data.copy()
```

In [3]:
```
# Create a copy of the original dataset
data_encoded = data.copy()

# Convert categorical values into numeric values
for column in data_encoded.columns:
    encoder = LabelEncoder()
    data_encoded[column] = encoder.fit_transform(
        data_encoded[column]
    )

# Display the encoded dataset
print(data_encoded.head())

# Separate features from the target variable
X = data_encoded.drop("class", axis=1)
y = data_encoded["class"]

print("\nFeatures:")
print(X.head())
print("\nTarget variable:")
```

```
print(y.head())

print("\nFeature shape:", X.shape)
print("Target shape:", y.shape)
```

```
   buying  maintenance  doors  persons  luggage_boot  safety  class
0       3            3      0        0             2       1      2
1       3            3      0        0             2       2      2
2       3            3      0        0             2       0      2
3       3            3      0        0             1       1      2
4       3            3      0        0             1       2      2

Features:
   buying  maintenance  doors  persons  luggage_boot  safety
0       3            3      0        0             2       1
1       3            3      0        0             2       2
2       3            3      0        0             2       0
3       3            3      0        0             1       1
4       3            3      0        0             1       2

Target variable:
0    2
1    2
2    2
3    2
4    2
Name: class, dtype: int64

Feature shape: (1728, 6)
Target shape: (1728,)
```

In [4]:
```
# Split the dataset using a 10% test set
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.1,
    random_state=42
)

# Display the sizes of the training and testing sets
print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
```

```
Training features: (1555, 6)
Testing features: (173, 6)
Training target: (1555,)
Testing target: (173,)
```

In [5]:
```
from sklearn.metrics import accuracy_score, classification_report
from sklearn.tree import DecisionTreeClassifier
# Create the Decision Tree classifier
model = DecisionTreeClassifier()

# Train the model
model.fit(X_train, y_train)
```

```
# Predict the test data
predictions = model.predict(X_test)

# Calculate the accuracy
accuracy = accuracy_score(y_test, predictions)

print("Decision Tree Accuracy:", accuracy)

# Display the classification report
print("\nClassification Report:")
print(classification_report(y_test, predictions))
```

```
Decision Tree Accuracy: 0.9595375722543352

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.93      0.94        44
           1       0.55      0.86      0.67         7
           2       1.00      1.00      1.00       112
           3       1.00      0.70      0.82        10

    accuracy                           0.96       173
   macro avg       0.87      0.87      0.86       173
weighted avg       0.97      0.96      0.96       173
```

In [6]:
```
from sklearn.metrics import confusion_matrix

# Generate the confusion matrix
cm = confusion_matrix(y_test, predictions)

print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[ 41   3   0   0]
 [  1   6   0   0]
 [  0   0 112   0]
 [  1   2   0   7]]
```

In [7]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Create the Random Forest classifier
rf_model = RandomForestClassifier(random_state=42)

# Train the model
rf_model.fit(X_train, y_train)

# Predict the test data
rf_predictions = rf_model.predict(X_test)

# Calculate accuracy
rf_accuracy = accuracy_score(y_test, rf_predictions)
```

```
print("Random Forest Accuracy:", rf_accuracy)

# Display the confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, rf_predictions))

# Display the classification report
print("\nClassification Report:")
print(classification_report(y_test, rf_predictions))
```

```
Random Forest Accuracy: 0.9479768786127167

Confusion Matrix:
[[ 37   6   1   0]
 [  1   6   0   0]
 [  0   0 112   0]
 [  1   0   0   9]]

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.84      0.89        44
           1       0.50      0.86      0.63         7
           2       0.99      1.00      1.00       112
           3       1.00      0.90      0.95        10

    accuracy                           0.95       173
   macro avg       0.86      0.90      0.87       173
weighted avg       0.96      0.95      0.95       173
```

In [8]:
```
# Get the feature importance values
feature_importance = pd.Series(
    rf_model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("Feature Importance:")
print(feature_importance)
```

```
Feature Importance:
safety          0.283351
persons         0.219349
buying          0.200236
maintenance     0.152656
luggage_boot    0.084786
doors           0.059623
dtype: float64
```

In [9]:
```
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Create the Extra Trees classifier
```

```
et_model = ExtraTreesClassifier(random_state=42)

# Train the model
et_model.fit(X_train, y_train)

# Predict the test data
et_predictions = et_model.predict(X_test)

# Calculate accuracy
et_accuracy = accuracy_score(y_test, et_predictions)

print("Extra Trees Accuracy:", et_accuracy)

# Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, et_predictions))

# Display classification report
print("\nClassification Report:")
print(classification_report(y_test, et_predictions))
```

```
Extra Trees Accuracy: 0.9595375722543352

Confusion Matrix:
[[ 39   1   4   0]
 [  1   6   0   0]
 [  0   0 112   0]
 [  1   0   0   9]]

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.89      0.92        44
           1       0.86      0.86      0.86         7
           2       0.97      1.00      0.98       112
           3       1.00      0.90      0.95        10

    accuracy                           0.96       173
   macro avg       0.94      0.91      0.93       173
weighted avg       0.96      0.96      0.96       173
```

In [10]:

```
# Compare the accuracy of all three classifiers
comparison = pd.DataFrame({
    "Model": [
        "Decision Tree",
        "Random Forest",
        "Extra Trees"
    ],
    "Accuracy": [
        accuracy_score(y_test, predictions),
        accuracy_score(y_test, rf_predictions),
        accuracy_score(y_test, et_predictions)
    ]
})
```

```
print(comparison.sort_values(
    by="Accuracy",
    ascending=False
).to_string(index=False))
```

```
       Model  Accuracy
Decision Tree  0.959538
  Extra Trees  0.959538
Random Forest  0.947977
```